CAPÍTULO 8

Resultados

Este capítulo presenta las mediciones obtenidas al evaluar logSguarDian, organizadas según los tres objetivos específicos del trabajo y la evaluación de interpretabilidad. Cada sección reporta los valores medidos, la condición bajo la cual se midieron y su fuente en el repositorio del proyecto. Las rutas de archivo se expresan relativas a la raíz de dicho repositorio. Cada medición se contrasta con su criterio de aceptación cuando existe uno; la interpretación de los valores se reserva para el capítulo de Discusión de resultados.

8.1. Resultados del objetivo específico 1: pipeline de ingeniería de características

El pipeline de ingeniería de características extrae 76 características por petición, de las cuales 69 alimentan al componente Random Forest y 63 al componente Isolation Forest (`training/results/v11_test_results.json`). Esta sección reporta la latencia del extractor en aislamiento y el sobrecosto absoluto del middleware completo en los entornos donde se midió.

8.1.1. Latencia del extractor en aislamiento

El Cuadro 1 presenta la latencia de la función `extractFeatureVector()` para cada una de las cinco clases de tráfico y para el tráfico mixto, en dos series: la llamada directa en el hilo principal y el ciclo de ida y vuelta a un *worker_ threads*, que es la ruta asíncrona del pipeline. Cada cifra es la mediana de tres corridas independientes del benchmark `benchmarks/extractor.bench.js` del repositorio `logSguarDian` (PR #97, fusionado en `develop`), del 9 de octubre de 2026. Cada corrida descartó 200 iteraciones de calentamiento, midió 2 000 iteraciones por *fixture* de forma serial y tomó el tiempo con alta resolución. El hardware fue un Apple M5 con macOS (Darwin 27.0.0, arm64) y Node.js v25.6.0.

La cifra que se compara con el criterio de 5 ms del objetivo específico 1 es el p95 mixto de la serie con *worker_ threads*, de 0.0273 ms. Ambas series son microbenchmarks en aislamiento: no incluyen el resto del middleware, la red ni la carga concurrente.

Fixture	Serie	p50 (ms)	p95 (ms)	p99 (ms)	Throughput (req/s)
`benign`	Directa (hilo principal)	0.0103	0.0169	0.0409	86 059
`sqli`	Directa (hilo principal)	0.0092	0.0120	0.0340	98 161
`xss`	Directa (hilo principal)	0.0102	0.0135	0.0340	92 380
`path_traversal`	Directa (hilo principal)	0.0076	0.0097	0.0302	122 212
`cmdi`	Directa (hilo principal)	0.0090	0.0102	0.0314	105 878
Mixto (*round-robin*)	Directa (hilo principal)	0.0097	0.0126	0.0350	98 383
`benign`	Ida y vuelta a *worker_ threads*	0.0213	0.0304	0.0467	43 960
`sqli`	Ida y vuelta a *worker_ threads*	0.0210	0.0275	0.0405	45 459
`xss`	Ida y vuelta a *worker_ threads*	0.0226	0.0305	0.0430	42 974
`path_traversal`	Ida y vuelta a *worker_ threads*	0.0162	0.0264	0.0377	54 884
`cmdi`	Ida y vuelta a *worker_ threads*	0.0212	0.0283	0.0405	45 180
Mixto (*round-robin*)	**Ida y vuelta a *worker_ threads***	**0.0206**	**0.0273**	**0.0413**	**46 635**

Cuadro 1: Latencia del extractor de características en aislamiento, por *fixture* y serie de ejecución. Fuente: `benchmarks/results/extractor-*.json` (tres corridas del 9 de octubre de 2026, commit `7b036bb` del código medido), rama `develop` del repositorio `logSguarDian`; mediana de las tres corridas por *fixture* y serie, a partir de los campos en nanosegundos. Elaboración propia.

Nota: una cifra anterior de 0.0082 ms, fechada el 20 de julio de 2026, no registra el hardware ni la versión de Node.js, por lo que no se usa como referencia en este trabajo.

8.1.2. Sobrecosto absoluto del middleware completo

El Cuadro 2 reúne las mediciones de latencia p95 con y sin el middleware activo en cada entorno donde se midió. Las filas difieren en entorno, escala de datos y compilación del middleware, por lo que cada una declara esas condiciones. Las filas de Docker más Postgres usan la aplicación de referencia (Express y Postgres) con el flujo de navegación benigno de Artillery (`attack-sim/artillery-baseline.yml`, 60 s, 20 usuarios virtuales por segundo con cuatro solicitudes por usuario, es decir, una tasa objetivo de 80 solicitudes por segundo). La fila de Node.js sobre WSL2 usa el benchmark de `packages/core/bench` (Artillery, 50 solicitudes por segundo, 30 s, 3 000 solicitudes por corrida, Node.js v22.5.1, Windows 11 con WSL2, 9 de julio de 2026) con resolución de 1 ms. La fila de microbenchmark mide una función Express sin aplicación detrás.

El Cuadro 2 reporta la latencia p95 con y sin middleware, y su diferencia Δp95 frente al límite de 15 ms fijado en el objetivo específico 3.

Como evidencia complementaria de inferencia, dos mediciones independientes documentan la latencia a nivel de componente, en contextos distintos que no deben compararse entre sí. Durante la instrumentación de la fase de *grace-window* (250 peticiones muestreadas), el tiempo medio de `session.run()` fue de 0.026 ms para Random Forest y 0.978 ms para Isolation Forest, con un tiempo medio de espera en cola (extracción y cola) de 0.038 ms y 0.052 ms, respectivamente (`docs/results.md`, líneas 445–460). Por separado, una ejecución de 60 s bajo carga sostenida midió el tiempo de inferencia por llamada de Isolation Forest en un rango de 2.99 ms a 3.22 ms de media (aproximadamente 2.9–3.2 ms, con p95 de 5.0–6.0 ms) a lo largo de diez segmentos cronológicos consecutivos, sin mostrar un patrón de crecimiento sostenido (`docs/results.md`, líneas 550–560; `docs/config2-latency-evaluation.md`, línea 452).

Entorno	Compilación del middleware	p95 sin middleware (ms)	p95 con middleware (ms)	Δp95 absoluto (ms)
Node.js sobre WSL2, Artillery	Versión de julio de 2026 (modo *monitor*)	1	1	0
Microbenchmark sintético (función Express)	Registro asíncrono sin ventana de gracia (PR #53)	0.116	0.300	0.184†
Aplicación real con Postgres, sin Docker	*Worker pool* con ventana de gracia, sin registro asíncrono	3	6	3†
Docker y Postgres, 10 filas	*Worker pool* con ventana de gracia, sin registro asíncrono	5.0	12.8	7.8†
Docker y Postgres, 10 filas	Diseño híbrido (ventana de gracia y cola de escritura por lotes)	≈5	≈12–14	≈7–9
Docker y Postgres, 2 010 filas	Diseño híbrido (ventana de gracia y cola de escritura por lotes)	13.67	23–28	9.33–14.33†

Cuadro 2: Latencia p95 con y sin middleware y diferencia absoluta, por entorno medido.

† Diferencia calculada como p95 con middleware menos p95 sin middleware; la fuente reporta ambos valores y no la diferencia. Las compilaciones con ventana de gracia corresponden a versiones anteriores del middleware; la versión vigente resuelve la respuesta sin esperar al Isolation Forest y actualiza el registro con su puntaje tardío (`packages/core/src/middleware.ts`, rama `develop`).
Fuente: `docs/results.md`, secciones A19 y F6.5; `docs/config2-latency-evaluation.md`, secciones Final latency y Config 2 Final Status. Elaboración propia.

8.2. Resultados del objetivo específico 2: modelo híbrido Random Forest e Isolation Forest

El conjunto de prueba bloqueado (57 481 filas) se leyó tres veces para la generación `rf_v11`/`if_v10`: el 24 de septiembre de 2026 (commit `3086318`), el 25 de septiembre de 2026 (commit `fc47a5b`) y el 26 de septiembre de 2026 (commit `03de50f`, mediante `training/evaluate_test.py`). Las cifras reportadas en esta sección corresponden a la lectura del 26 de septiembre de 2026. Se evaluaron los modelos `rf_v11` e `if_v10` en su forma ONNX (`rf.onnx` e `if.onnx`, ejecutados con `onnxruntime`) del repositorio de entrenamiento, y no los archivos `.pkl` ni el paquete publicado en npm. El archivo `training/splits/test.lock.sha256` coincidió con el *hash* registrado antes de ejecutar cualquier inferencia (`7486c258...4fd9`). Todos los valores de esta sección provienen de `training/results/v11_test_results.json`.

8.2.1. Random Forest

El Cuadro 3 presenta la precisión, el *recall*, el F1-score y el AUC-ROC de Random Forest por clase. El AUC-ROC se calculó con el método uno contra el resto (`roc_auc_score`, `multi_class='ovr'`). El F1-score macro fue de 0.9776, la exactitud global de 0.9932 y el F1-score ponderado de 0.9933. El AUC-ROC por clase estuvo entre 0.99899 (`path_traversal`) y 0.99998 (`benign`). Las métricas por clase se calculan por argmax de la probabilidad (`training/evaluate_test.py`, salida de etiquetas del modelo ONNX), sin el umbral de confianza de 0.35 con el que el middleware decide el bloqueo (`packages/core/src/middleware.ts`); no se calculó una métrica de prueba con la regla de decisión desplegada.

Clase	Precisión	*Recall*	F1-score	Soporte	AUC-ROC
`benign`	0.9984	0.9985	0.9984	14 886	0.99998
`cmdi`	0.9015	0.9475	0.9239	1 468	0.99901
`path_traversal`	0.9853	0.9798	0.9826	2 529	0.99899
`sqli`	0.9955	0.9956	0.9956	34 107	0.99982
`xss`	0.9950	0.9804	0.9877	4 491	0.99930
Promedio macro	**0.9751**	**0.9804**	**0.9776**	**57 481**	—

Cuadro 3: Métricas de Random Forest (`rf_v11`) por clase sobre el conjunto de prueba bloqueado.
Fuente: `training/results/v11_test_results.json`. Elaboración propia.

8.2.2. Isolation Forest

El Cuadro 4 presenta las métricas de Isolation Forest (`if_v10`) sobre el mismo conjunto de prueba. Una petición se considera detectada como anomalía cuando su puntaje queda por debajo del umbral de 0.004205941820353609, congelado en `training/models/if_v10_metadata.json` y no recalibrado contra el conjunto de prueba. El umbral de Isolation Forest pasó de 0.00807 (lectura del 24 de septiembre de 2026, con el modelo previo a ese reentrenamiento) a 0.00421 (lecturas del 25 y 26 de septiembre de 2026) tras reentrenar y recalibrar en validación. Las clases de ataque (42 595 filas) se tratan como positivas y la clase `benign` (14 886 filas) como negativa. La matriz de confusión resultante fue: 38 868 verdaderos positivos, 3 727 falsos negativos, 813 falsos positivos y 14 073 verdaderos negativos.

Métrica	Valor
Recall global	0.9125
Tasa de falsos positivos	0.0546
Precisión	0.9795
Recall en `cmdi`	0.9564
Recall en `path_traversal`	0.9968
Recall en `sqli`	0.8954
Recall en `xss`	0.9802

Cuadro 4: Métricas de Isolation Forest (`if_v10`) sobre el conjunto de prueba bloqueado.
Fuente: `training/results/v11_test_results.json`. Elaboración propia.

8.2.3. Paridad entre scikit-learn y ONNX

El Cuadro 5 presenta la diferencia máxima observada entre la salida de los modelos en scikit-learn y la de sus versiones ONNX, calculada sobre 1 000 muestras con *opset* 17.

Modelo	Magnitud comparada	Diferencia máxima
Random Forest (`rf_v11`)	Probabilidad por clase	1.01×10^{-7}
Isolation Forest (`if_v10`)	Puntaje de anomalía	2.42×10^{-7}

Cuadro 5: Diferencia máxima de salida entre scikit-learn y ONNX sobre 1 000 muestras. El reporte registra `parity_passed: true`.
Fuente: `training/models/parity_report.json`. Elaboración propia.

8.3. Resultados del objetivo específico 3: evaluación integrada en las configuraciones 1, 2, 3a y 3b

La evaluación integrada se ejecutó sobre una aplicación Express y Postgres deliberadamente vulnerable (`logSguarDian-vulnerable-project`), en cuatro configuraciones: Config 1, sin protección; Config 2, solo logSguarDian; Config 3a, solo *Web Application Firewall* (ModSecurity v3 con OWASP CRS en nivel de paranoia 1 y modo de bloqueo); y Config 3b, ModSecurity más logSguarDian en capas. En las configuraciones con logSguarDian, los modelos desplegados fueron `rf_v10` e `if_v9`, con `RF_THRESHOLD` de 0.35. La evaluación de Config 1, 2, 3a y 3b (rondas de julio-agosto de 2026) se ejecutó sobre la generación de modelos vigente en ese momento, `rf_v10`/`if_v9`. Las métricas de Random Forest e Isolation Forest reportadas en las secciones anteriores de este capítulo corresponden a la generación posterior, `rf_v11`/`if_v10`, resultado del reentrenamiento del 30 de agosto de 2026 (PR #66), cuyas métricas de prueba se re-derivaron en el PR #79. Las Configuraciones 2 y 3b se evaluaron con la generación vigente en ese momento, `rf_v10`/`if_v9`. La Configuración 1 y la Configuración 3a no usan los modelos de logSguarDian. La Configuración 2 se repitió después con `rf_v11`/`if_v10`, primera versión del 30 de agosto de 2026, como se reporta en las secciones 8.3.2 y 8.3.5, y con el par validado, como se reporta en la sección 8.3.6; la Configuración 3b no se repitió. Las rutas de archivo de esta sección se expresan relativas a `docs/` del repositorio del proyecto.

8.3.1. Config 1: aplicación sin protección

El Cuadro 6 resume los vectores de ataque ejecutados contra la aplicación sin ningún componente de seguridad, con la prueba de concepto de `curl` de cada vector y capturando la respuesta real.

Categoría	**Vectores probados**	**Vectores comprometidos**
Inyección SQL	Evasión de autenticación; exfiltración con `UNION`	2/2
XSS almacenado	Campo de biografía; contenido de publicación	2/2
Path traversal	Descarga de adjuntos	1/1
Inyección de comandos	Utilidad de administración `ping`	1/1
Total		**6/6**

Cuadro 6: Vectores de ataque comprometidos en Config 1, sin protección.

Fuente: `baseline-config1.md`, sección 5 (ejecutado el 24 de julio de 2026). Elaboración propia.

En el vector de inyección de comandos, `whoami` devolvió `root`.

8.3.2. Config 2: solo logSguarDian

El Cuadro 7 presenta la cobertura por categoría de Config 2 sobre 100 payloads por categoría (`attack-sim/payloads/*.json`), disparados contra el *endpoint* que corresponde a cada vulnerabilidad con el middleware activo en modo `block`. Se contabilizó como bloqueada la respuesta HTTP 403 del middleware.

Los 100 payloads por categoría de esta medición no son disjuntos del entrenamiento. Con la misma normalización de la sección 8.3.5, coinciden con alguna partición, por igualdad normalizada, 2 de 100 en `sqli`, 14 en `xss`, 4 en `path_traversal` y ninguno en `cmdi`, y por subcadena normalizada, 15, 33, 29 y 8, respectivamente. Algunas filas de las particiones no conservan el texto crudo, por lo que los conteos son cotas inferiores. El repositorio conserva

solo los valores agregados de esta medición, documentados el 6 de agosto de 2026, y no registra los resultados por payload ni el hash de los modelos, por lo que la cobertura no se recalculó sin las coincidencias.

Categoría	Bloqueados	Total	Cobertura
sqli	100	100	100.0 %
xss	100	100	100.0 %
path_traversal	93	100	93.0 %
cmdi	100	100	100.0 %

Cuadro 7: Cobertura de detección por categoría en Config 2, 100 payloads por categoría.
Fuente: `docs/config2-latency-evaluation.md` (secciones *Final coverage* y *Config 2 Final Status*) del repositorio `logSguarDian-vulnerable-project` (commit `944feb1`); valores agregados. Elaboración propia.

El recuento de eventos de bloqueo en la base de detección de logSguarDian (393) coincide con el total de payloads bloqueados del Cuadro 7, que el criterio de la sección anterior define como la respuesta HTTP 403 del middleware. En los escenarios normal y de volumen, las peticiones benignas de Artillery no recibieron ninguna respuesta 403 en las cinco repeticiones de cada condición (corrida de latencia de Linux, run 36276690236, `docs/evidence/latency-linux-2026-09-26.tgz` del repositorio `logSguarDian-vulnerable-project`, rama `main`). Cada repetición del escenario normal envió 6 300 solicitudes, incluido el calentamiento, de las cuales 4 725 respondieron 200 y 1 575 respondieron 302. En el escenario de volumen con middleware activo, 1 824 solicitudes en total agotaron el tiempo de espera entre las cinco repeticiones; un tiempo de espera no es una respuesta 403 y se reporta por separado.

Este resultado vale para el User-Agent por defecto de Artillery, `Artillery (https://artillery.io)`, una cadena corta que no representa a un navegador. Con cadenas de User-Agent de navegadores reales, el estudio `attack-sim/ua_fp_study.py` bloqueó 13 de 15 combinaciones de agente y ruta; solo Firefox en macOS evitó el bloqueo en `/login` y `/posts`.

Ese estudio corresponde al commit del 24 de septiembre de 2026, y su archivo de resultados no registra el modelo cargado. Por el contenido de `models/` en ese commit, el modelo es el par `rf_v10`/`if_v9`, mientras que la corrida de latencia de Linux usó el par `rf_v11`/`if_v10`, primera versión del 30 de agosto de 2026 (`a7c015f5...`/`3cf8e45c...`). Los dos resultados corresponden a modelos distintos y no se comparan directamente. En la condición de ataque no es posible aislar la tasa de 403 de las peticiones benignas, porque el informe agrega las respuestas por *endpoint* y `/profile` recibe tanto peticiones benignas como el ataque de XSS.

El Cuadro 8 reporta el incremento relativo del p95, definido como Δp95 expresado como porcentaje de la línea base sin middleware. Es un indicador complementario. El criterio vigente del objetivo específico 3 es el Δp95 absoluto de 15 ms del Cuadro 2. Cada medición usa el flujo benigno de Artillery (`attack-sim/artillery-baseline.yml`, 60 s, 20 usuarios virtuales por segundo con cuatro solicitudes por usuario) sobre Docker y Postgres, y las filas difieren en la compilación del middleware y en la escala de datos. Esas mediciones se documentaron entre el 31 de julio y el 6 de agosto de 2026, cuando el archivo tenía esa configuración; el cambio a 25 usuarios virtuales por segundo es del 24 de septiembre de 2026.

Las tres mediciones con 10 filas sembradas arrojaron un incremento relativo del p95 de +111.3 % a +265.8 %; la medición con 2 010 filas arrojó +70.2 %.

Experimento independiente de latencia en Linux nativo. El flujo manual `Latency`

Compilación del middleware	Datos sembrados	p95 sin middleware (ms)	p95 con middleware (ms)	Δp95 relativo
Un solo *worker* (`rf_v9`/`if_v8`)	10 filas	4.0	14.6	+265.8 %
Worker pool (`rf_v10`/`if_v9`), medición inicial	10 filas	5.0	10.6	+111.3 %
Worker pool (`rf_v10`/`if_v9`), umbral de IF corregido (6 corridas)	10 filas	5.0	12.8	+156.7 %
Diseño híbrido (ventana de gracia y cola de escritura por lotes)	2 010 filas	13.67	23–28	+70.2 %

Cuadro 8: Incremento relativo del p95 de Config 2 (indicador complementario), por compilación del middleware. Las generaciones medidas son `rf_v9`/`if_v8` y `rf_v10`/`if_v9`; no corresponden a `rf_v11`/`if_v10`.

Fuente: `docs/config2-latency-evaluation.md`, secciones *Latency Results*, *Comparison across iterations*, *Final latency*, *Definitive comparison* y *Config 2 Final Status*. Elaboración propia.

`Benchmark` del repositorio `logSguarDian-vulnerable-project` (ejecución 36276690236, commit `f15a9e8`, 26 de septiembre de 2026) se ejecutó en un runner `ubuntu-latest` con 2 vCPU y 7.8 GiB de memoria, con contenedores Docker sobre Linux y sin Docker Desktop/macOS. El Δp95 relativo fue de +485.1 % en el escenario normal, +278.1 % en el de ataque y +10 174.6 % en el de volumen con 2 010 filas sembradas. En términos absolutos, la diferencia entre el p95 con y sin middleware fue de 12.08 ms en el escenario normal (de 2.49 a 14.57 ms), 12.85 ms en el de ataque (de 4.62 a 17.47 ms) y 6 495.46 ms en el de volumen (de 63.84 a 6 559.30 ms). Frente al límite de 15 ms del objetivo específico 3, los escenarios normal y de ataque quedan por debajo y el de volumen lo supera ampliamente. Este experimento difiere de las mediciones anteriores en entorno, carga ofrecida, arnés de medición y procedencia de los artefactos. Se reporta por separado: no reemplaza ni refuta las cifras de +111.3 % a +265.8 % y +70.2 %, ni se promedia con ellas.

Cada corrida descarta 15 s de calentamiento y mide 60 s, con cinco repeticiones por condición. La tasa medida fue de 25 usuarios virtuales por segundo en los escenarios normal y de volumen, y de 30 en el de ataque, mediante `-overrides` en `attack-sim/run_latency_suite.sh`. El p95 de cada corrida es una aproximación calculada como promedio ponderado por conteo de los p95 de los intervalos de Artillery posteriores al calentamiento (`attack-sim/parse-latency-report.js`). Cada condición resume sus cinco repeticiones con la mediana de tres valores, tras descartar el mínimo y el máximo; no corresponde a un percentil exacto calculado sobre el conjunto agrupado de muestras.

En el escenario de volumen con middleware, 1 824 de 26 048 solicitudes (7.00 %, sumadas las cinco repeticiones) terminaron con `ERR_SOCKET_TIMEOUT`: 1 804 en `/login` y 20 en `/posts`. La línea base de volumen no registró errores. Las solicitudes con timeout no tienen tiempo de respuesta registrado y no entran en el p95.

Los modelos de esta ejecución son `rf.onnx` (SHA-256 `a7c015f5...`) e `if.onnx` (`3cf8e45c...`), que se encuentran en el directorio `models/` de la aplicación vulnerable (commit `f15a9e8`) y no forman parte del paquete publicado en npm (`logsguardian@0.1.0`, par A), que contiene `rf_v10`/`if_v9`. Corresponden al build `rf_v11`/`if_v10` del 30 de agosto de 2026 (commit `734c24f` del repositorio logSguarDian), el mismo que usaron las ejecuciones de Config 2 del 9 de septiembre de 2026. No son el par canónico posterior al PR #79 (`rf.onnx 25b407e6...`, `if.onnx 5be28ee8...`), que es el que declara el campo `artifact_checksums` de `training/results/v11_test_results.json` y con el que se calcularon las métricas de la sección del objetivo específico 2. La ejecución de Linux no se repitió con ese par. Fuente: `cybersecurity-objectives-compliance.md`, sección OE3.2 y

sus notas metodológica y de procedencia del modelo, en el repositorio logSguarDian.

8.3.3. Config 3a y Config 3b: corpus pequeño

En las Rondas 1 y 2, ejecutadas con el *stack* en capas y ModSecurity en modo de bloqueo, el corpus fue de 23 payloads: los 9 casos de ataque de las Configs 1 y 2 y 14 variantes diseñadas para evadir las reglas de firmas de CRS. ModSecurity respondió 403 a los 9 casos originales y a las 14 variantes (23/23), y ninguna petición llegó a la aplicación (`docs/config3b-results.md`, secciones *Round 1* y *Round 2*).

En la Ronda 3, con `MODSEC_RULE_ENGINE` en `DetectionOnly`, de modo que CRS registra pero no bloquea y toda petición llega a la aplicación, los 23 payloads se reejecutaron contra logSguarDian. La auditoría de CRS registró que habría bloqueado los 23. El resultado inicial fue de 16/23 bloqueados por logSguarDian (la cifra original del documento fuente, 21/23, resultó aritméticamente inconsistente con el detalle de 7 casos no detectados que el propio documento reporta; se corrigió a 16/23, consistente con 16+7=23, sin re-medición porque los datos crudos de esa ronda ya no existen); los 7 no detectados eran de XSS y se habían enviado como `multipart/form-data`. Tras montar un *parser* multipart global antes del middleware, 23/23 devolvieron `verdict: block`, confirmado en `detection_events`. Dos payloads (traversal de rutas y una inyección de comandos) se bloquearon con una clase asignada distinta de la esperada (`config3b-results.md`, sección *Round 3*).

8.3.4. Config 3a y Config 3b: corpus grande

La Ronda 4 ejecutó las cuatro configuraciones contra 590 payloads de SecLists (rama `master`), deduplicados y muestreados aleatoriamente con semilla 42 hasta un máximo de 200 por categoría: 77 de `sqli`, 113 de `xss`, 200 de `path_traversal` y 200 de `cmdi`. El Cuadro 9 presenta los payloads bloqueados por configuración y categoría.

Categoría	Config 1 (sin protección)	Config 2 (solo logSguarDian)	Config 3a (solo WAF, PL1)	Config 3b (en capas)
`sqli` (77)	0/77 (0.0 %)	76/77 (98.7 %)	66/77 (85.7 %)	77/77 (100.0 %)
`xss` (113)	0/113 (0.0 %)	110/113 (97.3 %)	110/113 (97.3 %)	113/113 (100.0 %)
`path_traversal` (200)	0/200 (0.0 %)	197/200 (98.5 %)	174/200 (87.0 %)	197/200 (98.5 %)
`cmdi` (200)	0/200 (0.0 %)	200/200 (100.0 %)	200/200 (100.0 %)	200/200 (100.0 %)
Total (590)	**0/590 (0.0 %)**	**583/590 (98.8 %)**	**550/590 (93.2 %)**	**587/590 (99.5 %)**

Cuadro 9: Payloads bloqueados por configuración y categoría sobre el corpus de 590 payloads de SecLists.

Fuente: `docs/config3b-results.md`, sección *Round 4*. Elaboración propia.

De los 40 ataques que CRS en nivel de paranoia 1 no bloqueó en Config 3a (590 menos 550), la atribución de los bloqueos de Config 3b por firma de respuesta (el encabezado `X-Powered-By: Express` aparece solo en bloqueos de la capa de aplicación) registró 37 capturados por logSguarDian tras pasar el WAF. Tres payloads pasaron ambas capas: las cadenas `~/.gtkrc`, `/proc/filesystems` y `~/.atfp_history`, de la categoría `path_traversal`.

8.3.5. Solapamiento del corpus de 590 payloads con el entrenamiento

Los 590 payloads de la evaluación integral provienen de SecLists. Los 200 de inyección de comandos son las líneas del archivo `command-injection-commix.txt`, del que también provienen las 2 455 filas de la fuente `seclists_cmdi` (1 719 en entrenamiento, 368 en validación y 368 en prueba). Con igualdad de la cadena normalizada (dígitos sustituidos por `N` y marcadores de seis letras mayúsculas por `CANARY` en `cmdi`; decodificación URL, minúsculas y

dígitos sustituidos en las demás categorías), los 200 payloads de `cmdi` coinciden con alguna partición. Por texto literal, 63 de ellos aparecen en alguna partición: 46 en entrenamiento, 8 en validación y 9 en prueba, sin que ninguno aparezca en dos particiones. En `xss`, 7 de 113 payloads coinciden por igualdad normalizada; en `sqli`, ninguno de 77; en `path_traversal`, ninguno de 200. Al excluir las coincidencias por igualdad normalizada, la cobertura de logSguarDian en Config 2 fue de 98.7 % en `sqli`, 97.2 % en `xss` y 98.5 % en `path_traversal`, frente a 98.7 %, 97.3 % y 98.5 % con el corpus completo. La coincidencia por subcadena produce 26 payloads de `sqli`, 13 de `xss` y los 200 de `cmdi`, y constituye un límite superior, no una medida de solapamiento. En `cmdi` todo el corpus coincide, por lo que la cobertura de 100 % en esa categoría no constituye evidencia de generalización a payloads de otra procedencia.

Config 2 se ejecutó también sobre los mismos 590 payloads con dos generaciones de modelo; el cuadro anterior corresponde a la primera. Con `rf_v10/if_v9` (`attack-sim/results_config2.json`, commit `af61eba` del repositorio `logSguarDian-vulnerable-project`) se bloquearon 583 de 590 payloads. Con `rf_v11/if_v10` (build del 30 de agosto de 2026, `attack-sim/results_config2-rfv11.json`, commit `14c0e15`) también se bloquearon 583 de 590. Al comparar por identificador, 10 payloads cambiaron de estado: `sqli` pasó de 76 a 77 bloqueados (`sqli_31` pasó a bloquearse), `xss` de 110 a 111 (`xss_12`), `path_traversal` se mantuvo en 197 con tres payloads que dejaron de bloquearse (`path_traversal_13`, `_127` y `_186`) y tres que empezaron a bloquearse (`_6`, `_36` y `_69`), y `cmdi` bajó de 200 a 198 (`cmdi_88` y `cmdi_118` dejaron de bloquearse). El archivo `results_config2-rfv11.json` no registra el modelo utilizado; su atribución al build del 30 de agosto de 2026 (`rf.onnx a7c015f5...`, `if.onnx 3cf8e45c...`) se apoya en el mensaje del commit `14c0e15`, que describe el reemplazo de `models/rf.onnx` e `models/if.onnx` por el build `rf_v11/if_v10` antes de esa ejecución. El archivo `results_config2.json` (590 payloads) corresponde a `rf_v10/if_v9`: los blobs de `models/rf.onnx` e `models/if.onnx` en `af61eba` tienen los hashes `18083725...` y `5de374a7...` de ese par, y el reporte de paridad `training/models/parity_report.json` del commit `dc2aaa65` (repositorio logSguarDian) declara `rf_v10/if_v9`.

8.3.6. Evaluación sobre un corpus sin solapamiento (Ronda 5)

Para acotar el solapamiento descrito en la subsección anterior se construyó un corpus de 608 solicitudes. Contiene 200 payloads de inyección SQL generados con sqlmap, 200 de path traversal y 200 de inyección de comandos (`cmdi`). Estos dos últimos conjuntos se produjeron con generadores propios deterministas, con semilla 20260923, no derivados de SecLists. El corpus incluye además 8 marcadores de `xss` escritos a mano; no entran en la cobertura, por lo que esta categoría no se evaluó en esta ronda.

Antes de incluir cada candidato se descartó toda coincidencia exacta del campo completo con el conjunto de entrenamiento. El filtro descartó 3 de 400 candidatos de path traversal, todos por coincidencia con `owasp_logs.jsonl`, y ninguno de los 1 995 candidatos de sqli ni de los 400 de cmdi (`attack-sim/exclusion_report.json`). El solapamiento con las particiones se midió con `training/leakage_check.py` del repositorio `logSguarDian` (commit 22550ea), ejecutado en solo lectura contra `train`, `val` y `test`. Se usan dos criterios. El primero, (a), es la igualdad normalizada del campo completo: un payload coincide si su forma normalizada es igual a la de algún campo `path`, `query` o `body` de la partición. El segundo, (b), es la regla de subcadena de `round4_vs_split`: un payload coincide si aparece como subcadena de algún campo, con y sin normalización. La regla (b) es permisiva, por lo que sus cifras son un límite superior y no una medida de solapamiento.

El Cuadro 10 presenta el número de payloads de cada categoría que coinciden con cada partición, por criterio.

Categoría	Criterio	Train	Val	Test	Unión
`sqli`	(a) igualdad normalizada	4	1	2	4
`sqli`	(b) subcadena normalizada	4	1	2	4
`sqli`	(b) subcadena literal	0	0	0	0
`path_traversal`	(a) igualdad normalizada	0	0	0	0
`path_traversal`	(b) subcadena normalizada	6	5	5	7
`path_traversal`	(b) subcadena literal	5	2	5	5
`cmdi`	(a) igualdad normalizada	0	0	0	0
`cmdi`	(b) subcadena normalizada	1	1	1	2
`cmdi`	(b) subcadena literal	0	0	0	0

Cuadro 10: Payloads de 200 por categoría que coinciden con cada partición, según el criterio de solapamiento. Unión: payloads que coinciden con al menos una partición.

Fuente: `training/leakage_check.py` (commit 22550ea) sobre `training/splits/` del repositorio `logSguarDian`, y `attack-sim/large_corpus_requests.json`. Elaboración propia.

Con el criterio (a), 4 de los 200 payloads de sqli coinciden con alguna partición y ninguno de path traversal ni de cmdi. Con el criterio (b), path traversal alcanza 7 coincidencias en la unión y cmdi 2; por ser un límite superior, estas cifras no deben leerse como solapamiento confirmado.

El modelo validado tiene una cadena de procedencia verificable. El tarball de validación, que no se publicó en el registro npm, se construyó desde el commit 22550ea de `develop` del repositorio `logSguarDian`, con SHA-256 `cf1238deb33383975e12058de71c4423b22d7c5bcba5cf47241c0d0f712b0416`. Sus modelos, par C, son `rf.onnx` (`25b407e6...`) e `if.onnx` (`5be28ee8...`), y las sumas de verificación se comprobaron dentro del contenedor. La construcción quedó registrada en el commit 4d928b5 del repositorio `logSguarDian-vulnerable-project`. La repetición de Config 2 con ese tarball se ejecutó el 9 de octubre de 2026 (`attack-sim/results_config2-develop-tarball-20261009.json`), con los mismos 200 payloads por categoría. El archivo de resultados no registra el modelo; esa atribución consta en `docs/config3b-results.md` y en dicho commit.

El Cuadro 11 presenta la cobertura de Config 2 sobre este corpus con ambos pares de modelos.

Categoría	`rf_v10/if_v9` (paquete publicado)	`rf_v11/if_v10` (validado)
`sqli`	200/200 (100.0 %)	199/200 (99.5 %)
`path_traversal`	170/200 (85.0 %)	190/200 (95.0 %)
`cmdi`	183/200 (91.5 %)	192/200 (96.0 %)

Cuadro 11: Cobertura de Config 2 sobre el corpus de la Ronda 5, con 200 solicitudes por categoría, por par de modelos.

Fuente: `attack-sim/results_config2.json` en el commit a3aada0 y `attack-sim/results_config2-develop-tarball-20261009.json`, del repositorio `logSguarDian-vulnerable-project`. Elaboración propia.

Con `rf_v10`/`if_v9`, los intervalos de Wilson al 95 % son 98.1–100.0 % en sqli, 79.4–89.3 % en path traversal y 86.8–94.6 % en cmdi. Con `rf_v11`/`if_v10` son 97.2–99.9 %, 91.0–97.3 % y 92.3–98.0 %, en el mismo orden. Las tres categorías superan el 80 % del objetivo específico 3 con ambos pares de modelos, aunque el límite inferior del intervalo de path traversal con `rf_v10`/`if_v9` queda por debajo de ese umbral.

Al comparar por identificador, 26 payloads de path traversal pasaron de no bloqueados a bloqueados y 6 en sentido inverso; en cmdi, 16 pasaron a bloqueados y 7 en sentido inverso; en sqli, ninguno pasó a bloqueado y 1 en sentido inverso. En la corrida del paquete, `cmdi_94` terminó por *timeout*; en la corrida validada, `sqli_75` devolvió HTTP 500. Ambas entradas se contaron como no bloqueadas.

Esta evaluación tiene limitaciones que acotan su alcance. Se realizó una sola corrida, con 200 solicitudes por categoría. Los corpus de path traversal y cmdi son sintéticos, y la categoría `xss` no tiene un corpus limpio, por lo que su cobertura permanece en el corpus de 590 payloads. A la fecha de redacción de este capítulo (10 de octubre de 2026), las configuraciones 3a y 3b no se repitieron, de modo que la comparación con el WAF también permanece en ese corpus. Los resultados de la Ronda 5 con `rf_v10`/`if_v9` se conocieron el 24 de septiembre de 2026, y el modelo `rf_v11`/`if_v10` se reconstruyó el 26 de septiembre de 2026. El historial no vincula ambos hechos, por lo que esta no es una evaluación estrictamente ciega del modelo reconstruido. Por último, este capítulo no evalúa la causa de la diferencia de path traversal entre ambas corridas (de 85.0 % a 95.0 %).

8.3.7. Falsos positivos en aplicaciones de terceros

En la aplicación vulnerable del proyecto, con el User-Agent por defecto de Artillery, ninguna petición benigna de los escenarios normal y de volumen recibió una respuesta 403 (cifras en la sección de resultados de Config 2, obtenidas con el par `rf_v11`/`if_v10`, primera versión del 30 de agosto de 2026, `a7c015f5...`/`3cf8e45c...`); el resultado cambia con User-Agent de navegador, como se describe a continuación. En aplicaciones de terceros la proporción de peticiones benignas bloqueadas fue alta. En OWASP Juice Shop (modo `monitor`, solo peticiones benignas), 303 de 312 peticiones con User-Agent de navegador (97.1 %) y 123 de 312 con `curl` (39.4 %) recibieron el veredicto de bloqueo. Con un User-Agent de navegador se bloquearon 37 de 40 peticiones benignas en dvws-node (92.5 %) y 5 de 5 en DVNA, medición preliminar. En node-api-goat se bloquearon 50 de 50 valores enviados a una sola ruta, todos con el User-Agent de Chrome 128 (`evidence/node-api-goat/run-attacks.js`); el chequeo de cinco peticiones con ese mismo User-Agent dio 4 de 5. En dvws-node, el User-Agent por defecto de Node bloqueó 30 de 40 (75 %).

La medición en Juice Shop usó la copia local de Juice Shop (par C), no el paquete publicado en npm. Esa copia contiene `rf_v11` e `if_v10` (SHA-256 `25b407e6...` y `5be28ee8...`), los mismos que el repositorio de entrenamiento conserva en `training/models/`. Para dvna, node-api-goat y dvws-node, el paquete y el modelo exactos de cada medición no constan en el repositorio.

8.3.8. Consumo de memoria y CPU

La memoria y la CPU se midieron con `docker stats` sobre el contenedor de la aplicación, no sobre un proceso. La CPU se expresa como porcentaje del contenedor, de modo que 100 % equivale a un núcleo. La medición corresponde a la misma ejecución del experimento de latencia en Linux nativo (ejecución 36276690236), en un runner `ubuntu-latest` con 2 vCPU y 7.8 GiB de memoria, con cinco repeticiones por condición. La ventana A, primaria, excluye los primeros 15 s de cada repetición, que corresponden al calentamiento a baja carga; la ventana B (repetición completa) sí lo incluye.

El Cuadro 12 usa la ventana A, que descarta los primeros 15 s de cada repetición. La ventana B, que conserva la repetición completa con el arranque, da una ΔCPU relativa

Escenario	Memoria baseline / active (MiB)	ΔMemoria (MiB)	CPU baseline / active (% de un núcleo)	ΔCPU (pp / relativo)
Ataque	63.2 / 394.8	331.9	16.1 / 101.3	85.1 pp / 529.6 %
Normal	65.2 / 397.5	332.0	14.5 / 103.3	88.9 pp / 614.4 %
Volumen	83.3 / 459.1	375.9	82.3 / 115.3	33.0 pp / 40.1 %

Cuadro 12: Memoria y CPU del contenedor con y sin middleware, por escenario (ventana A).

Fuente: `docs/evidence/resources_summary.json`, rama `docs/latency-linux-evidence`, ventana A. Medias por condición; ΔMemoria y ΔCPU son la media de las diferencias pareadas por repetición. Elaboración propia.

mayor en los tres escenarios (541.3 %, 668.9 % y 57.5 %). Ambas ventanas llevan a la misma conclusión: ningún escenario cumple el criterio de CPU.

El criterio de memoria del protocolo es de 150 MB de memoria adicional, equivalentes a unos 143 MiB. Se aplica al incremento ΔMemoria que añade el middleware, no a la memoria absoluta del contenedor. Los incrementos medios fueron de 331.9 MiB en ataque, 332.0 MiB en normal y 375.9 MiB en volumen. Los tres superan el límite, y cada uno es más del doble de los 143 MiB, por lo que el criterio no se cumple en este experimento. El repositorio del proyecto revisó internamente este límite a 300 MB; esta tesis mantiene el criterio del protocolo. Los tres incrementos (331.9, 332.0 y 375.9 MiB, es decir, 348.0, 348.1 y 394.2 MB) superan también ese valor revisado. El incremento es casi el mismo en los escenarios normal y de ataque (331.9 y 332.0 MiB).

El límite se fijó antes de medir la memoria nativa de `onnxruntime-node` y pensando en instancias de 1 GB (Droplet). El contenedor completo con el middleware activo ocupó entre 394.8 y 459.1 MiB (ataque, normal y volumen: 394.8, 397.5 y 459.1 MiB), y no se probó en una instancia de 1 GB, por lo que el cumplimiento en ese entorno no se verificó. El criterio de CPU aprobado es un ΔCPU relativo de hasta 15 % bajo carga normal y de ataque. Ambos escenarios lo superan (529.6 % y 614.4 %), y el escenario de volumen también (40.1 %).

Estas cifras corresponden a una sola corrida (ejecución 36276690236, commit medido f15a9e8) en un runner de 2 vCPU, con cinco repeticiones por condición. Se midió el contenedor completo con `docker stats`, sin separar sus procesos, por lo que el costo no se atribuye a ningún componente. El archivo de datos tiene SHA-256 `b96d9014...2290187cec`.

La memoria con el middleware activo es prácticamente la misma en los escenarios normal y de ataque, y se mantiene en un rango similar entre repeticiones. La corrida de monitoreo continuo de 41.9 minutos (374 muestras) registró un pico de 494.8 MiB, medido sobre el contenedor.

No se midió el costo por componente. La configuración crea tres `worker_threads`: dos workers de Isolation Forest (`IF_POOL_SIZE` = 2) y uno de Random Forest. Las cifras de este experimento corresponden al conjunto del middleware y no pueden atribuirse a ninguno de sus componentes.

Estas cifras no son comparables con el RSS de 245.11 MB del benchmark aislado de `logSguarDian` (`benchmarks/onnx-memory-pool.bench.js`). Ese benchmark no incluye carga HTTP ni contenedor y no tiene línea base sin middleware.

8.4. Resultados de interpretabilidad: análisis SHAP

El análisis de interpretabilidad se calculó con `shap.TreeExplainer` sobre el modelo `rf_v11` (`training/models/rf_v11.pkl`) y la partición de validación (`training/splits/val.parquet`), con las características de entrada del modelo seleccionadas por nombre. El *script* es `training/results/shap_analysis.py`, y los reportes y figuras de esta sección están en

training/results/. La importancia global de una característica para una clase es la media del valor absoluto de su valor SHAP sobre la partición de validación, para la salida de esa clase.

8.4.1. Importancia global por clase

El Cuadro 13 presenta, para cada una de las cinco clases, las tres características de mayor importancia global y la posición (entre las diez primeras) de ua_length y ua_present. Las Figuras 1 a 5 muestran las diez características de mayor importancia de cada clase. La Figura 2 muestra la importancia SHAP global de la clase sqli. La Figura 3 muestra la de la clase xss. La Figura 4 muestra la de la clase path_traversal.

Clase	Tres características de mayor importancia (media |SHAP|)	Posición de ua_length	Posición de ua_present
benign	ua_length (0.08659); ua_present (0.05109); path_depth (0.03777)	1	2
sqli	ua_length (0.07342); sqli_keyword_density (0.05159); ua_present (0.03683)	1	3
xss	alert_function_present (0.03458); xss_marker_density (0.0274); html_tag_count (0.02689)	5	8
path_traversal	traversal_sequence_count (0.05172); special_char_ratio (0.03731); ua_length (0.03207)	3	4
cmdi	method_is_get (0.02554); ua_length (0.02122); subshell_count (0.01584)	2	9

Cuadro 13: Características de mayor importancia SHAP global por clase de rf_v11 y posición de las características de *User-Agent*.

Fuente: training/results/shap_global_report.json. Elaboración propia.

En la lista de diez características de cmdi no figuran shell_command_count ni pipe_count.

Las características del User-Agent figuran entre las de mayor contribución. ua_length ocupa el primer lugar en benign (0.0866) y en sqli (0.0734), el segundo en cmdi y el tercero en path_traversal; ua_present ocupa el segundo lugar en benign y el tercero en sqli. En el caso local de sqli del Cuadro 14, ua_length aporta +0.1136, la mayor contribución individual. Solo el 0.4 % de las filas benignas tiene un User-Agent largo (más de 40 caracteres), frente a entre 33.3 % (xss) y 97.7 % (path_traversal) en las clases de ataque.

8.4.2. Explicaciones locales

Las Figuras 6 y 7 presentan la explicación local de dos peticiones GET /profile, una sin *User-Agent* y otra con él, y la Figura 8 la de una petición de sqli del conjunto de prueba de extremo a extremo (e2e/fixtures/test_payloads.jsonl). En cada caso el valor SHAP corresponde a la clase de mayor probabilidad predicha, y el valor base es 0.2000. El Cuadro 14 reporta los valores SHAP de ua_present y ua_length en los tres casos.

8.5. Limitaciones de los resultados

La regla R2 no se cumplió para la generación vigente. El conjunto de prueba se leyó tres veces para rf_v11/if_v10, el 24, el 25 y el 26 de septiembre de 2026, y las cifras de

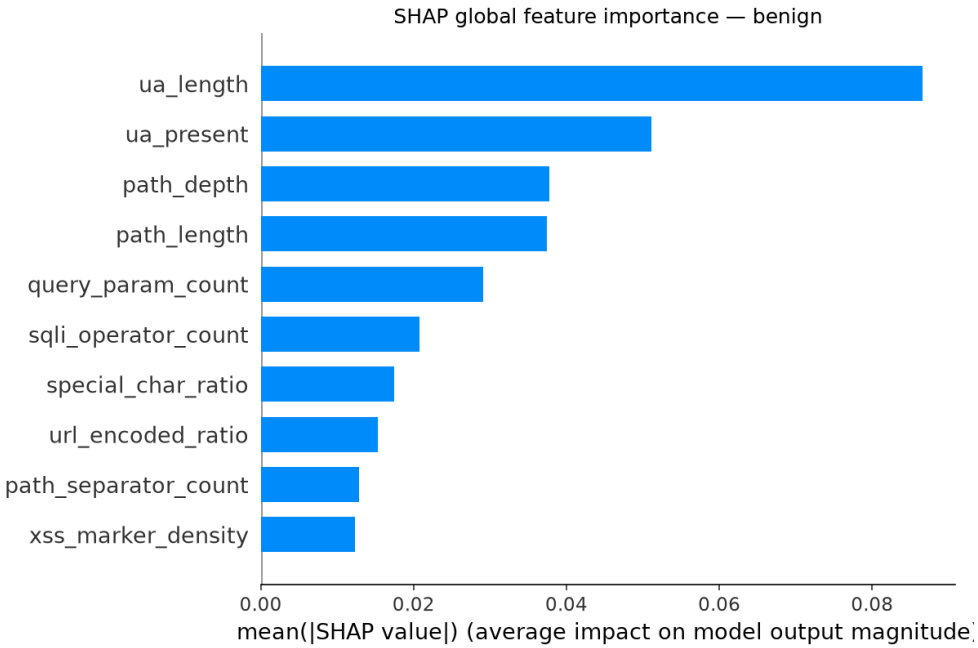


Figura 1: Importancia SHAP global, clase `benign`.

Fuente: elaboración propia, `training/results/shap_analysis.py`.

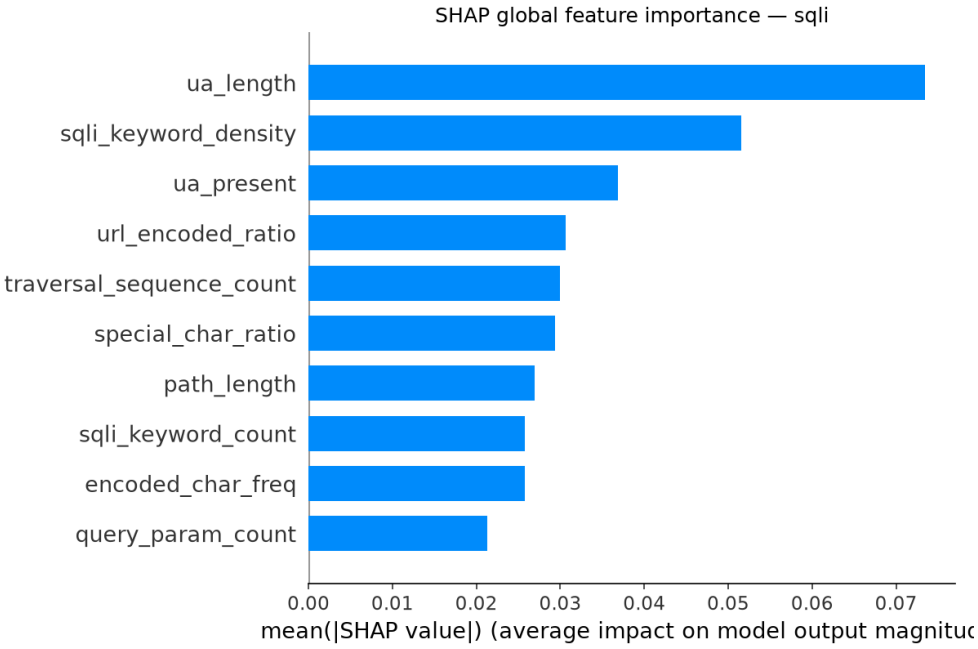


Figura 2: Importancia SHAP global, clase `sqli`.

Fuente: elaboración propia, `training/results/shap_analysis.py`.

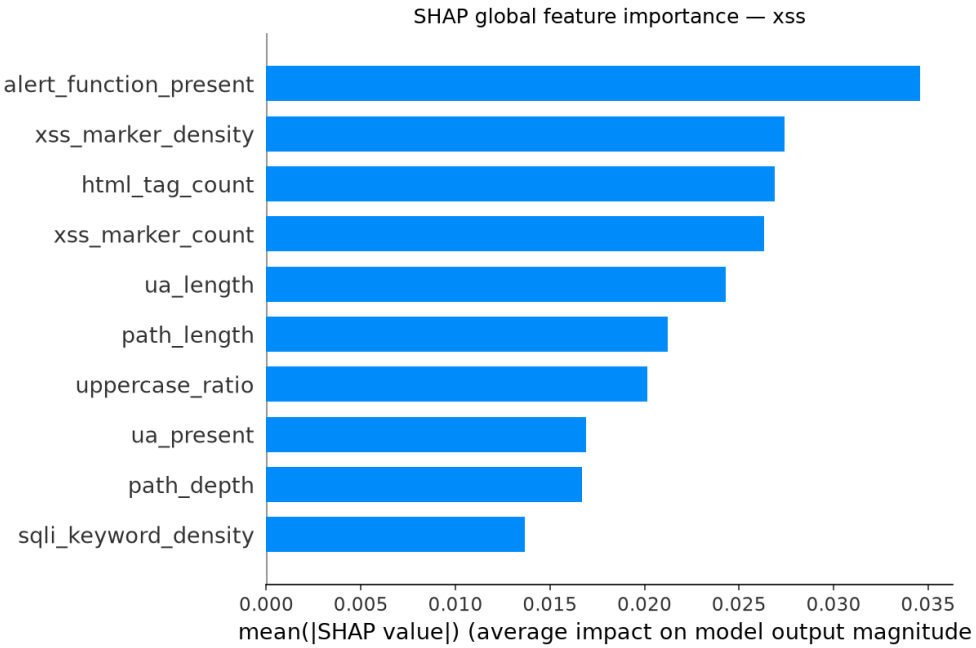


Figura 3: Importancia SHAP global, clase `xss`.

Fuente: elaboración propia, `training/results/shap_analysis.py`.

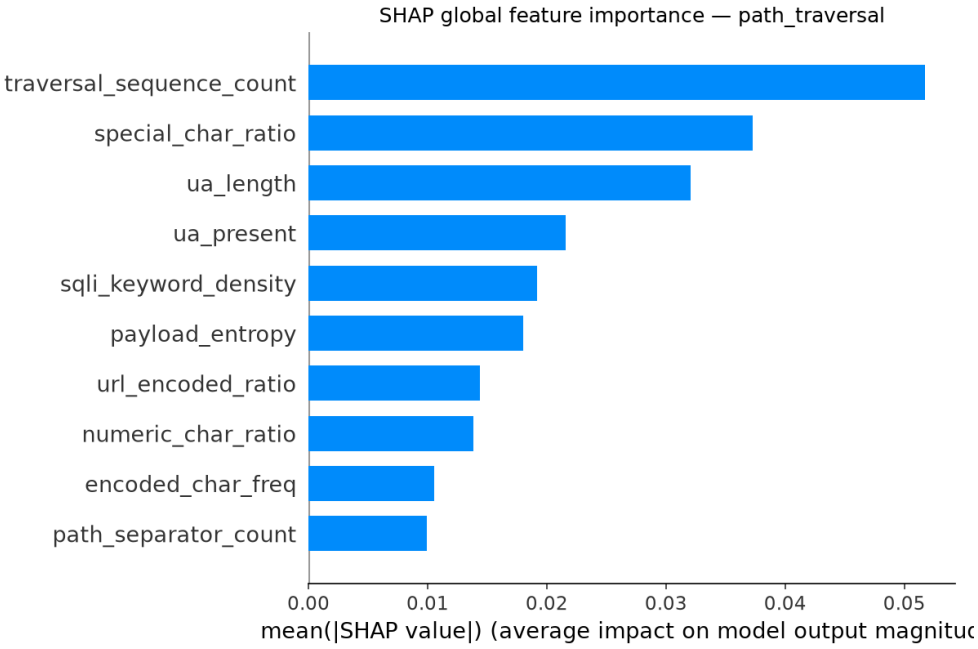


Figura 4: Importancia SHAP global, clase `path_traversal`.

Fuente: elaboración propia, `training/results/shap_analysis.py`.

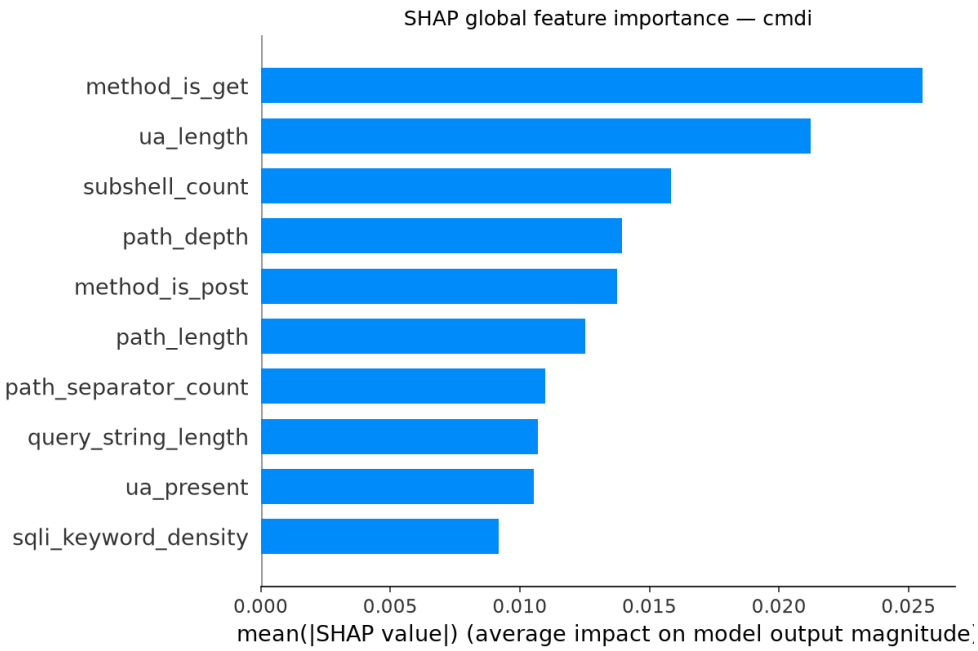


Figura 5: Importancia SHAP global, clase `cmdi`.

Fuente: elaboración propia, `training/results/shap_analysis.py`.

Caso	Clase predicha (probabilidad)	Valor de `ua_present`; SHAP	Valor de `ua_length`; SHAP	Mayor contribución (SHAP)
`GET /profile` sin *User-Agent*	`xss` (0.650)	0; +0.0596	0; +0.0678	`path_length` (+0.1008)
`GET /profile` con *User-Agent*	`benign` (0.533)	1; −0.0498	75; −0.0327	`special_char_ratio` (+0.0566)
Petición de `sqli` del conjunto de extremo a extremo	`sqli` (0.999)	1; +0.0485	68; +0.1136	`ua_length` (+0.1136)

Cuadro 14: Valores SHAP locales de las características de *User-Agent* y mayor contribución, para la clase predicha de cada caso.

Fuente: valores SHAP recalculados con `training/results/shap_analysis.py` sobre `rf_v11`; figuras en `training/results/shap_local_*.png`. Elaboración propia.

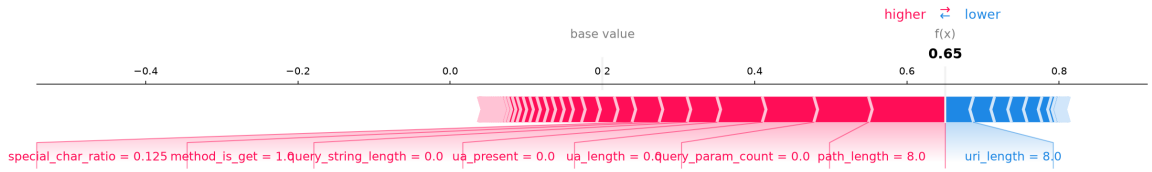


Figura 6: Explicación local de `GET /profile` sin *User-Agent* (clase `xss`).

Fuente: elaboración propia, `training/results/shap_analysis.py`.

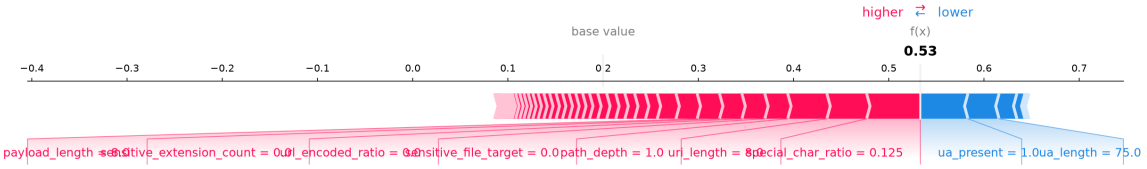


Figura 7: Explicación local de `GET /profile` con *User-Agent* (clase `benign`).
Fuente: elaboración propia, `training/results/shap_analysis.py`.

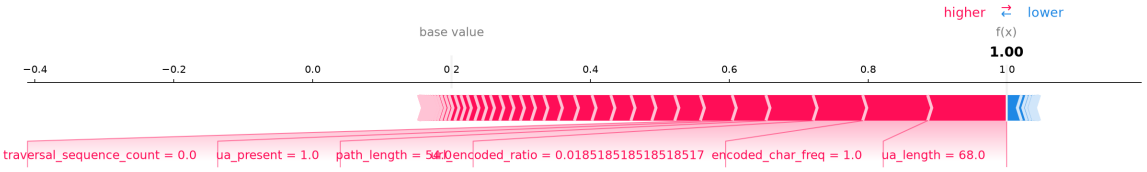


Figura 8: Explicación local de una petición de `sqli` detectada (clase `sqli`).
Fuente: elaboración propia, `training/results/shap_analysis.py`.

este capítulo corresponden a la lectura del 26 de septiembre. La sección de Recolección, procesamiento y validación de datos describe las tres lecturas.

Los objetivos de calibración de Isolation Forest se endurecieron después de fallos observados en el conjunto de prueba. En `if_v2`, la tasa de falsos positivos fue de 0.1011 frente a un límite de 0.10; en `if_v5`, de 0.0811 frente a un objetivo interno de 0.08. El límite formal del objetivo específico 2 se mantiene en una tasa de falsos positivos de 10 %.

El umbral de 0.35 de Random Forest se calibró sobre validación para `rf_v3` y no se recalibró para `rf_v11`. Como se indicó en la sección de resultados del objetivo específico 2, las métricas de prueba de Random Forest se calculan por argmax y no con este umbral.

De las 1 468 filas de `cmdi` del conjunto de prueba, 261 (17.8 %) son sintéticas. Su generador no está versionado en el repositorio, por lo que esas filas no pueden regenerarse a partir del código del proyecto.

En el conjunto de entrenamiento, CAPEC aporta el 81.3 % de las filas de `sqli` y el 85.8 % de las de `path_traversal`, y modsec-learn el 69.6 % de las filas `benign`. Además, 1 221 de las 6 853 filas de `cmdi` del entrenamiento (17.8 %) son sintéticas.

La fuente de los datos restaurados el 23 de septiembre de 2026 no está documentada. Además, `unified.jsonl` contiene 383 083 filas y `features.parquet` contiene 383 202; la diferencia de 119 filas corresponde a las filas de telemetría curada del pipeline de MLOps, que `unify.py` excluye y `ct_pipeline.py` añade después.

Las pruebas en aplicaciones de terceros muestran una tasa alta de bloqueo de tráfico benigno, y la contribución del User-Agent al modelo, descrita en la sección SHAP. Este trabajo no evaluó el modelo con un corpus benigno alternativo. El conjunto de tráfico benigno no cubre la diversidad de clientes HTTP que cabría esperar en un entorno real.

La contribución de Isolation Forest a la detección no se midió. Por diseño, este componente no bloquea peticiones; su salida solo enriquece los registros.

Los datos crudos de latencia de Config 2 no constan en el repositorio: los re-

portes de Artillery de los experimentos en Docker no están versionados, porque `attack-sim/results/` está excluido en `.gitignore`. Las cifras de Config 2 se documentan en `docs/config2-latency-evaluation.md` del repositorio `logSguarDian-vulnerable-project`. Los reportes del experimento de latencia en Linux nativo se conservan en el artefacto de GitHub Actions `latency-benchmark-results` (id 10917957439, ejecución 36276690236), copiado a `docs/evidence/latency-linux-2026-09-26.tgz` del repositorio `logSguarDian-vulnerable-project` (rama `main`).

En cuanto a memoria y CPU, las cifras y el alcance de la medición se describen en la subsección 8.3.8. El indicador heurístico `possibleLeak` de la corrida de monitoreo de 41.9 minutos compara las mitades de una serie que mezcla condiciones distintas, por lo que no es concluyente. Además, 60 s por repetición no bastan para descartar una fuga de memoria.

La confusión entre `cmdi` y `sqli` en técnicas de tiempo ciego es la limitación de etiquetado más marcada. En la re-medición del 9 de septiembre de 2026 con `rf_v11`/`if_v10`, los 200 payloads de inyección de comandos del corpus se etiquetaron todos como `sqli` y ninguno como `cmdi`; 198 se bloquearon y 2 se registraron como `pass_anomaly`. El corpus usa oráculos de tiempo (`sleep`) y comparaciones aritméticas, y el hallazgo afecta la clase reportada en el triaje, no la decisión de bloqueo. Como los 200 payloads coinciden con filas de las particiones de entrenamiento, validación o prueba (sección 8.3.5), la medición no constituye evidencia de generalización. Una evaluación sobre un corpus sin solapamiento, con las cautelas que allí se describen, se presenta en la subsección 8.3.6. Fuentes: `docs/config2-results-v1.md`, §5.1, del repositorio `logSguarDian`, y `attack-sim/cmdi_sqli_attribution_rfv11_-2026-09-09.json` del repositorio `logSguarDian-vulnerable-project` (commit 14c0e15).

Los 100 payloads por categoría de la cobertura de Config 2 tampoco son disjuntos del entrenamiento, y la cobertura no se recalculó sin esas coincidencias; véase la subsección 8.3.2.

El conjunto de prueba se vectoriza a partir de las cadenas almacenadas, mientras que el middleware reconstruye la query con `URLSearchParams` antes de extraer las características, lo que modifica algunas de ellas para la misma petición (por ejemplo, `quote_count` pasa de 1 a 0 con el payload `' OR 1=1--`). Las métricas de prueba de este capítulo no incluyen ese efecto, que no se midió.

A la fecha de redacción de este capítulo (10 de octubre de 2026), el paquete publicado en npm (`logsguardian@0.1.0`, par A) contiene `rf_v10`/`if_v9`. Esos modelos son los que se evaluaron de extremo a extremo en las Configuraciones 1 a 3b. En esa fecha, las métricas de modelo de este capítulo corresponden a `rf_v11`/`if_v10`, disponibles en el repositorio y no publicados.